

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA0504

COURSE NAME: Database Management Systems

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE (SID, S Name, CID, C Name, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R (A, B, C, D, E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE (Emp-ID, Emp-Name, Dept-ID, Dept-Name, Manager-ID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R (A, B, C, D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER (TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5

9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

*I **SANTHOSH** / 192511470 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Santhosh

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1) For the relation STUDENT (SID, S-Name, CID, C-Name, Instructor, Grade), identify all functional dependencies and determine the candidate key(s).

Given Relation

STUDENT (SID, S-Name, CID, C-Name, Instructor, Grade)

Assume:

- SID = Student ID
- S-Name = Student Name
- CID = Course ID
- C-Name = Course Name
- Instructor = Course Instructor
- Grade = Grade obtained by student

Functional Dependencies

The functional dependencies are:

SID $\rightarrow$ S-Name

A Student ID uniquely determines the student's name.

CID $\rightarrow$ C-Name

A Course ID uniquely determines the course name.

CID $\rightarrow$ Instructor

A Course ID uniquely determines the instructor.

(SID, CID) $\rightarrow$ Grade

A student's grade depends on both the student and the course.

Therefore:

SID $\rightarrow$ S-Name

CID $\rightarrow$ C-Name

CID $\rightarrow$ Instructor

SID, CID $\rightarrow$ Grade

Determine Candidate Key

We need a key that determines all attributes.

Take:

(SID, CID)+

Using the functional dependencies:

SID → S-Name

CID → C-Name

CID → Instructor

SID, CID → Grade

Therefore:

(SID, CID)+

= {SID, CID, S-Name, C-Name, Instructor, Grade}

This contains all attributes of the relation.

Neither SID nor CID alone can determine all attributes:

SID+ = {SID, S-Name}

CID+ = {CID, C-Name, Instructor}

Therefore, both are required.

Functional Dependencies:

SID → S-Name

CID → C-Name

CID → Instructor

(SID, CID) → Grade

Candidate Key:

(SID, CID)

Primary Key can be:

PRIMARY KEY (SID, CID)

This is a composite key because it consists of two attributes.

CREATE TABLE AND INSERT

```
mysql> CREATE TABLE Student (  
-> S_ID INT,  
-> SName VARCHAR(50),  
-> CID INT,  
-> CName VARCHAR(50),  
-> Instructor VARCHAR(50),  
-> Grade CHAR(2),  
-> PRIMARY KEY (S_ID, CID)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO Student  
-> VALUES  
-> (101, 'Rahul', 501, 'DBMS', 'Dr. Kumar', 'A'),  
-> (101, 'Rahul', 502, 'Python', 'Dr. Priya', 'B+'),  
-> (102, 'Anjali', 501, 'DBMS', 'Dr. Kumar', 'A+'),  
-> (102, 'Anjali', 503, 'Java', 'Dr. Ravi', 'B'),  
-> (103, 'Arun', 502, 'Python', 'Dr. Priya', 'A');  
Query OK, 5 rows affected (0.02 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM Student;  
+-----+-----+-----+-----+-----+-----+  
| S_ID | SName | CID | CName | Instructor | Grade |  
+-----+-----+-----+-----+-----+-----+  
| 101 | Rahul | 501 | DBMS | Dr. Kumar | A |  
| 101 | Rahul | 502 | Python | Dr. Priya | B+ |  
| 102 | Anjali | 501 | DBMS | Dr. Kumar | A+ |  
| 102 | Anjali | 503 | Java | Dr. Ravi | B |  
| 103 | Arun | 502 | Python | Dr. Priya | A |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.01 sec)
```

Functional Dependencies

Check $S_ID \rightarrow S\text{-Name}$

```
mysql> SELECT S_ID, SName  
-> FROM Student  
-> GROUP BY S_ID, SName  
-> ORDER BY S_ID;  
  
+-----+-----+  
| S_ID | SName |  
+-----+-----+  
| 101 | Rahul |  
| 102 | Anjali |  
| 103 | Arun |  
+-----+-----+  
3 rows in set (0.01 sec)
```

Student-Course-Grade Combination

```
mysql> SELECT S_ID, CID, Grade
-> FROM Student
-> ORDER BY S_ID, CID;
```

S_ID	CID	Grade
101	501	A
101	502	B+
102	501	A+
102	503	B
103	502	A

5 rows in set (0.00 sec)

Check CID → C-Name, Instructor

```
mysql> SELECT CID, CName, Instructor
-> FROM Student
-> GROUP BY CID, CName, Instructor
-> ORDER BY CID;
```

CID	CName	Instructor
501	DBMS	Dr. Kumar
502	Python	Dr. Priya
503	Java	Dr. Ravi

3 rows in set (0.00 sec)

JUSTIFICATION: The candidate key is (S_ID, CID) because together they uniquely identify every student-course record and determine all attributes of the relation.

2) Determine the process of generating a minimal relation into 1NF with step-by-step justification and example.

What is 1NF?

- A relation is in First Normal Form (1NF) when:
- Every attribute contains atomic (single) values.
- There are no repeating groups.
- Each row represents a single record.
- Each column contains values from the same domain.

Example: Relation Not in 1NF

Consider:

STUDENT

SID	S-Name	Courses
101	Rahul	DBMS, Python
102	Priya	Java, DBMS, C++
103	Arun	Python

CREATE AND INSERT DATA

```
mysql> CREATE TABLE Student_Unnormalized (
  ->     S_ID INT,
  ->     SName VARCHAR(50),
  ->     Courses VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student_Unnormalized
  -> VALUES
  -> (101, 'Rahul', 'DBMS, Python'),
  -> (102, 'Priya', 'Java, DBMS'),
  -> (103, 'Arun', 'Python, C++');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student_Unnormalized;
+-----+-----+-----+
| S_ID | SName | Courses |
+-----+-----+-----+
| 101  | Rahul | DBMS, Python |
| 102  | Priya | Java, DBMS |
| 103  | Arun  | Python, C++ |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

1NF TABLE AND INSERT ATOMIC VALUES

```
mysql> CREATE TABLE Student_Course (
->     S_ID INT,
->     SName VARCHAR(50),
->     Course VARCHAR(50),
->     PRIMARY KEY (S_ID, Course)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student_Course
-> VALUES
-> (101, 'Rahul', 'DBMS'),
-> (101, 'Rahul', 'Python'),
-> (102, 'Priya', 'Java'),
-> (102, 'Priya', 'DBMS'),
-> (103, 'Arun', 'Python'),
-> (103, 'Arun', 'C++');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student_Course;
+-----+-----+-----+
| S_ID | SName | Course |
+-----+-----+-----+
| 101  | Rahul | DBMS   |
| 101  | Rahul | Python |
| 102  | Priya | DBMS   |
| 102  | Priya | Java   |
| 103  | Arun  | C++    |
| 103  | Arun  | Python |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

JUSTIFICATION:

To convert a relation into 1NF, identify multi-valued or repeating attributes, split them into individual atomic values, create separate rows for each value, and identify an appropriate key.

3) Given relation R(A, B, C, D, E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it into 2NF eliminating partial dependencies.

Given

Relation:

R (A, B, C, D, E)

Functional dependencies:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Step 1: Find the Candidate Key

Start with A:

$A^+ = \{A\}$

Using:

$A \rightarrow BC$

we get:

$A^+ = \{A, B, C\}$

Using:

$BC \rightarrow D$

we get:

$A^+ = \{A, B, C, D\}$

Using:

$D \rightarrow E$

we get:

$A^+ = \{A, B, C, D, E\}$

Therefore:

A is the candidate key

Step 2: Check for Partial Dependency

- A partial dependency occurs when a non-key attribute depends on only a part of a composite candidate key.
- Here, the candidate key is only A, which is a single attribute.
- Therefore, there cannot be any partial dependency.

So:

R (A, B, C, D, E) is already in 2NF

```
mysql> CREATE TABLE R (
  ->   A INT PRIMARY KEY,
  ->   B VARCHAR(20),
  ->   C VARCHAR(20),
  ->   D VARCHAR(20),
  ->   E VARCHAR(20)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO R
  -> VALUES
  -> (1, 'B1', 'C1', 'D1', 'E1'),
  -> (2, 'B2', 'C2', 'D2', 'E2'),
  -> (3, 'B3', 'C3', 'D3', 'E3'),
  -> (4, 'B4', 'C4', 'D4', 'E4'),
  -> (5, 'B5', 'C5', 'D5', 'E5');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R;
+----+-----+-----+-----+-----+
| A  | B    | C    | D    | E    |
+----+-----+-----+-----+-----+
| 1  | B1   | C1   | D1   | E1   |
| 2  | B2   | C2   | D2   | E2   |
| 3  | B3   | C3   | D3   | E3   |
| 4  | B4   | C4   | D4   | E4   |
| 5  | B5   | C5   | D5   | E5   |
+----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT A, B, C
-> FROM R
-> ORDER BY A;
```

A	B	C
1	B1	C1
2	B2	C2
3	B3	C3
4	B4	C4
5	B5	C5

5 rows in set (0.00 sec)

```
mysql> SELECT B, C, D
-> FROM R
-> ORDER BY B, C;
```

B	C	D
B1	C1	D1
B2	C2	D2
B3	C3	D3
B4	C4	D4
B5	C5	D5

5 rows in set (0.00 sec)

```
mysql> SELECT D, E
-> FROM R
-> ORDER BY D;
```

D	E
D1	E1
D2	E2
D3	E3
D4	E4
D5	E5

5 rows in set (0.00 sec)

Final Answer

Item	Result
Relation	R(A, B, C, D, E)
FDs	$A \rightarrow BC, BC \rightarrow D, D \rightarrow E$
Candidate Key	A
Partial Dependency	None
2NF	Already in 2NF
Decomposition	Not required for 2NF

JUSTIFICATION: Although there is no partial dependency, there is a transitive dependency ($A \rightarrow D \rightarrow E$), which becomes relevant when checking 3NF, not 2NF.

4) Apply 3NF decomposition on the relation EMPLOYEE (Emp-ID, Emp-Name, Dept-ID, Dept-Name, Manager-ID) with transitive dependencies.

Given Relation

EMPLOYEE (Emp-ID, Emp-Name, Dept-ID, Dept-Name, Manager-ID)

Assume the functional dependencies are:

Emp-ID $\rightarrow$ Emp-Name, Dept-ID, Manager-ID

Dept-ID $\rightarrow$ Dept-Name

Therefore, we have a transitive dependency:

Emp-ID $\rightarrow$ Dept-ID $\rightarrow$ Dept-Name

Here, Dept-Name depends on Dept-ID, and Dept-ID depends on Emp-ID.

Step 1: Find Candidate Key

Find the closure of Emp-ID:

$$\begin{aligned} &\text{Emp-ID}^+ \\ &= \{\text{Emp-ID}\} \end{aligned}$$

Using:

$$\text{Emp-ID} \rightarrow \text{Emp-Name, Dept-ID, Manager-ID}$$

we get:

$$\text{Emp-ID}^+ = \{\text{Emp-ID, Emp-Name, Dept-ID, Manager-ID}\}$$

Using:

$$\text{Dept-ID} \rightarrow \text{Dept-Name}$$

we get:

$$\text{Emp-ID}^+ = \{\text{Emp-ID, Emp-Name, Dept-ID, Dept-Name, Manager-ID}\}$$

Therefore, Emp-ID determines all attributes.

Emp-ID=Candidate Key

Step 2: Identify Transitive Dependency

We have:

Emp-ID $\rightarrow$ Dept-ID

Dept-ID $\rightarrow$ Dept-Name

Therefore:

Emp-ID $\rightarrow$ Dept-Name

through Dept-ID.

This is a transitive dependency.

For 3NF, we remove this dependency by separating department information into another table.

Step 3: Decompose into 3NF

Relation 1 — EMPLOYEE

EMPLOYEE (Emp-ID, Emp-Name, Dept-ID, Manager-ID)

Primary Key:

Emp-ID

Relation 2 — DEPARTMENT

DEPARTMENT (Dept-ID, Dept-Name)

Primary Key:

Dept-ID

Dept-ID in EMPLOYEE becomes a Foreign Key referencing DEPARTMENT.

CREATE AND INSERT DATA

```
mysql> CREATE TABLE Employee_Original (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(50),  
->     DeptID INT,  
->     DeptName VARCHAR(50),  
->     ManagerID INT  
-> );
```

Query OK, 0 rows affected (0.04 sec)

```
mysql> INSERT INTO Employee_Original  
-> VALUES  
-> (101, 'Rahul', 10, 'CSE', 201),  
-> (102, 'Priya', 20, 'ECE', 202),  
-> (103, 'Arun', 10, 'CSE', 201),  
-> (104, 'Kiran', 30, 'IT', 203),  
-> (105, 'Sneha', 20, 'ECE', 202);
```

Query OK, 5 rows affected (0.01 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Employee_Original;
```

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Rahul	10	CSE	201
102	Priya	20	ECE	202
103	Arun	10	CSE	201
104	Kiran	30	IT	203
105	Sneha	20	ECE	202

5 rows in set (0.00 sec)

EMPLOYEE TABLE

```
mysql> CREATE TABLE Employee (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(50),  
->     DeptID INT,  
->     ManagerID INT,  
->     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
-> );
```

Query OK, 0 rows affected (0.05 sec)

```
mysql> INSERT INTO Employee  
-> VALUES  
-> (101, 'Rahul', 10, 201),  
-> (102, 'Priya', 20, 202),  
-> (103, 'Arun', 10, 201),  
-> (104, 'Kiran', 30, 203),  
-> (105, 'Sneha', 20, 202);
```

Query OK, 5 rows affected (0.01 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Employee;
```

EmpID	EmpName	DeptID	ManagerID
101	Rahul	10	201
102	Priya	20	202
103	Arun	10	201
104	Kiran	30	203
105	Sneha	20	202

5 rows in set (0.00 sec)

DEPARTMENT TABLE

```
mysql> CREATE TABLE Department (  
->     DeptID INT PRIMARY KEY,  
->     DeptName VARCHAR(50)  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO Department  
-> VALUES  
-> (10, 'CSE'),  
-> (20, 'ECE'),  
-> (30, 'IT');
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Department;
```

DeptID	DeptName
10	CSE
20	ECE
30	IT

3 rows in set (0.00 sec)

JOIN:

```
mysql> SELECT  
->     Employee.EmpID,  
->     Employee.EmpName,  
->     Employee.DeptID,  
->     Department.DeptName,  
->     Employee.ManagerID  
-> FROM Employee  
-> JOIN Department  
-> ON Employee.DeptID = Department.DeptID;
```

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Rahul	10	CSE	201
103	Arun	10	CSE	201
102	Priya	20	ECE	202
105	Sneha	20	ECE	202
104	Kiran	30	IT	203

5 rows in set (0.01 sec)

5) Verify whether R (A, B, C, D) with FDs $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose into BCNF.

Given

Relation:

R (A, B, C, D)

Functional dependencies:

$AB \rightarrow C$

$C \rightarrow D$

$D \rightarrow A$

Step 1: Find Candidate Keys

Check AB

$(AB)^+ = \{A, B\}$

Using:

$AB \rightarrow C$

we get:

$\{A, B, C\}$

Using:

$C \rightarrow D$

we get:

$(AB)^+ = \{A, B, C, D\}$

Therefore, AB is a candidate key.

Check BC

Start with:

BC

Using:

$C \rightarrow D$

we get:

B, C, D

Using:

$D \rightarrow A$

we get:

$(BC)^+ = \{A, B, C, D\}$

Therefore, BC is also a candidate key.

Check BD

Using:

$D \rightarrow A$

we get:

B, D, A

Using:

$AB \rightarrow C$

we get:

$(BD)^+ = \{A, B, C, D\}$

Therefore, BD is also a candidate key.

Candidate Keys

AB, BC, BD

Step 2: Check BCNF

A relation is in BCNF if, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a super key.

Check each FD.

FD 1

$AB \rightarrow C$

AB is a candidate key.

- **Satisfies BCNF.**

FD 2

$C \rightarrow D$

Is C a super key? **No.**

$C^+ = \{C, D, A\}$

It does not contain B.

- **Violates BCNF.**

FD 3

$D \rightarrow A$

D is not a super key.

- **Violates BCNF.**

Therefore:

R is NOT in BCNF

Step 3: BCNF Decomposition

Use the violating dependency:

$$C \rightarrow D$$

Decompose into:

Relation 1

R1(C, D)

Relation 2

R2(A, B, C)

So:

R 1(C, D)

R 2 (A, B, C)

CREATE AND INSERT DATA

```
mysql> CREATE TABLE R (  
->     A INT,  
->     B INT,  
->     C INT,  
->     D INT,  
->     PRIMARY KEY (A, B)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO R  
-> VALUES  
-> (1, 10, 100, 1000),  
-> (2, 20, 200, 2000),  
-> (3, 30, 100, 1000),  
-> (4, 40, 300, 3000),  
-> (5, 50, 200, 2000);  
Query OK, 5 rows affected (0.01 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM R;  
+----+-----+-----+-----+  
| A | B | C | D |  
+----+-----+-----+-----+  
| 1 | 10 | 100 | 1000 |  
| 2 | 20 | 200 | 2000 |  
| 3 | 30 | 100 | 1000 |  
| 4 | 40 | 300 | 3000 |  
| 5 | 50 | 200 | 2000 |  
+----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

DECOMPOSE INTO BCNF

R1

```
mysql> CREATE TABLE R1 (  
->      C INT PRIMARY KEY,  
->      D INT  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO R1  
-> SELECT DISTINCT C, D  
-> FROM R;  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM R1;  
+-----+-----+  
| C      | D      |  
+-----+-----+  
| 100    | 1000   |  
| 200    | 2000   |  
| 300    | 3000   |  
+-----+-----+  
3 rows in set (0.00 sec)
```

R2

```
mysql> CREATE TABLE R2 (  
->      A INT,  
->      B INT,  
->      C INT,  
->      PRIMARY KEY (A, B)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> INSERT INTO R2  
-> SELECT DISTINCT A, B, C  
-> FROM R;  
Query OK, 5 rows affected (0.01 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM R2;  
+-----+-----+-----+  
| A      | B      | C      |  
+-----+-----+-----+  
| 1      | 10     | 100    |  
| 2      | 20     | 200    |  
| 3      | 30     | 100    |  
| 4      | 40     | 300    |  
| 5      | 50     | 200    |  
+-----+-----+-----+  
5 rows in set (0.00 sec)
```

JOIN

```
mysql> SELECT R2.A, R2.B, R2.C, R1.D
-> FROM R2
-> JOIN R1
-> ON R2.C = R1.C;
```

A	B	C	D
1	10	100	1000
2	20	200	2000
3	30	100	1000
4	40	300	3000
5	50	200	2000

5 rows in set (0.01 sec)

Justification

The functional dependencies $C \rightarrow D$ and $D \rightarrow A$ violate BCNF because C and D are not super keys of the original relation. Decomposing the relation using the violating dependency $C \rightarrow D$ produces R1(C, D) and R2(A, B, C), eliminating the BCNF violation while preserving the required information.

6) Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Consider:

R (A, B, C)

with functional dependency:

$A \rightarrow B$

Decompose R into:

R1(A,B)

and

R2(A,C)

We need to check whether this decomposition is lossless.

Step 1: Chase Table

Create a row containing attributes:

	A	B	C
R ₁	a	b	b ₂
R ₂	a	b ₂	c

Because both relations contain A, compare their A values.

They are equal:

$$\mathbf{A = a}$$

The functional dependency is:

$$\mathbf{A \rightarrow B}$$

Therefore, because the A values are the same, the B value becomes the same.

The resulting row becomes:

	A	B	C
Result	a	b	c

Since we obtain a row containing the original distinguished symbols:

a, b, c

the decomposition is lossless.

CREATE AND INSERT VALUES

```
mysql> CREATE TABLE R (
->     A INT PRIMARY KEY,
->     B VARCHAR(20),
->     C VARCHAR(20)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO R
-> VALUES
-> (1, 'B1', 'C1'),
-> (2, 'B2', 'C2'),
-> (3, 'B3', 'C3'),
-> (4, 'B4', 'C4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R;
+----+-----+-----+
| A  | B    | C    |
+----+-----+-----+
| 1  | B1   | C1   |
| 2  | B2   | C2   |
| 3  | B3   | C3   |
| 4  | B4   | C4   |
+----+-----+-----+
4 rows in set (0.00 sec)
```

CREATE R1 and R2

```
mysql> CREATE TABLE R1 (  
->     A INT PRIMARY KEY,  
->     B VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO R1  
-> SELECT A, B  
-> FROM R;  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE R2 (  
->     A INT PRIMARY KEY,  
->     C VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO R2  
-> SELECT A, C  
-> FROM R;  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM R1;  
+----+-----+  
| A  | B    |  
+----+-----+  
| 1  | B1   |  
| 2  | B2   |  
| 3  | B3   |  
| 4  | B4   |  
+----+-----+  
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM R2;  
+----+-----+  
| A  | C    |  
+----+-----+  
| 1  | C1   |  
| 2  | C2   |  
| 3  | C3   |  
| 4  | C4   |  
+----+-----+
```

```
mysql> SELECT R1.A, R1.B, R2.C
      -> FROM R1
      -> JOIN R2
      -> ON R1.A = R2.A;
```

A	B	C
1	B1	C1
2	B2	C2
3	B3	C3
4	B4	C4

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) AS Original_Rows
      -> FROM R;
```

Original_Rows
4

```
1 row in set (0.01 sec)
```

```
mysql> SELECT COUNT(*) AS Reconstructed_Rows
      -> FROM R1
      -> JOIN R2
      -> ON R1.A = R2.A;
```

Reconstructed_Rows
4

```
1 row in set (0.00 sec)
```

Justification

The decomposition is lossless because the common attribute A functionally determines B ($A \rightarrow B$). The Chase algorithm therefore produces the required distinguished row, and the SQL JOIN of R1 and R2 reconstructs exactly the original relation without losing or generating extra tuples.

7) Identify the multi-valued dependencies in TEACHER (TID, Subject, Hobby) and decompose it into 4NF.

Given relation:

TEACHER (TID, Subject, Hobby)

Assume:

- TID = Teacher ID
- Subject = Subjects taught by the teacher
- Hobby = Hobbies of the teacher

Suppose a teacher can teach multiple subjects and can have multiple hobbies, and these two are independent of each other.

Therefore, the multi-valued dependencies are:

$TID \twoheadrightarrow Subject$

and

$TID \twoheadrightarrow Hobby$

The symbol $\twoheadrightarrow$ represents a multi-valued dependency.

Step 1: Example of the Problem

Suppose teacher 101 teaches:

- DBMS
- Python
- Reading
- Music

The original relation needs combinations of the independent values:

TID	Subject	Hobby
101	DBMS	Reading
101	DBMS	Music
101	Python	Reading
101	Python	Music

This creates redundancy because every subject gets combined with every hobby.

Step 2: Identify MVDs

We have:

$TID \twoheadrightarrow Subject$

$TID \twoheadrightarrow Hobby$

These are multi-valued dependencies because one teacher can have multiple independent subjects and multiple independent hobbies.

Step 3: Why 4NF is Required

A relation is in Fourth Normal Form (4NF) if:

For every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X must be a super key.

Here:

TID $\rightarrow\rightarrow$ Subject

TID $\rightarrow\rightarrow$ Hobby

But TID alone does not uniquely identify a single row because one teacher can have multiple subjects and hobbies.

Therefore, the original relation violates 4NF.

Step 4: Decompose into 4NF

Separate the independent multi-valued attributes.

Relation 1

TEACHER_SUBJECT (TID, Subject)

Relation 2

TEACHER_HOBBY (TID, Hobby)

Now the independent multi-valued facts are stored separately.

```
mysql> CREATE TABLE Teacher (
->     TID INT,
->     Subject VARCHAR(50),
->     Hobby VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Teacher
-> VALUES
-> (101, 'DBMS', 'Reading'),
-> (101, 'DBMS', 'Music'),
-> (101, 'Python', 'Reading'),
-> (101, 'Python', 'Music'),
-> (102, 'Java', 'Cricket'),
-> (102, 'Java', 'Music');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Teacher;
+-----+-----+-----+
| TID | Subject | Hobby |
+-----+-----+-----+
| 101 | DBMS    | Reading |
| 101 | DBMS    | Music   |
| 101 | Python  | Reading |
| 101 | Python  | Music   |
| 102 | Java    | Cricket |
| 102 | Java    | Music   |
+-----+-----+-----+
6 rows in set (0.00 sec)
```



```

mysql> CREATE TABLE Teacher_Subject (
->     TID INT,
->     Subject VARCHAR(50),
->     PRIMARY KEY (TID, Subject)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Teacher_Subject
-> VALUES
-> (101, 'DBMS'),
-> (101, 'Python'),
-> (102, 'Java');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Teacher_Hobby (
->     TID INT,
->     Hobby VARCHAR(50),
->     PRIMARY KEY (TID, Hobby)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Teacher_Hobby
-> VALUES
-> (101, 'Reading'),
-> (101, 'Music'),
-> (102, 'Cricket'),
-> (102, 'Music');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

```

```

mysql> SELECT * FROM Teacher_Subject;
+-----+-----+
| TID | Subject |
+-----+-----+
| 101 | DBMS    |
| 101 | Python  |
| 102 | Java    |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Teacher_Hobby;
+-----+-----+
| TID | Hobby   |
+-----+-----+
| 101 | Music   |
| 101 | Reading |
| 102 | Cricket |
| 102 | Music   |
+-----+-----+
4 rows in set (0.00 sec)

```

```
mysql> SELECT
      ->     TS.TID,
      ->     TS.Subject,
      ->     TH.Hobby
      -> FROM Teacher_Subject TS
      -> JOIN Teacher_Hobby TH
      -> ON TS.TID = TH.TID;
```

TID	Subject	Hobby
101	DBMS	Music
101	DBMS	Reading
101	Python	Music
101	Python	Reading
102	Java	Cricket
102	Java	Music

6 rows in set (0.00 sec)

Justification

The decomposition is justified because subjects and hobbies are independent multi-valued facts about a teacher. Storing them in separate relations avoids unnecessary combinations and eliminates the non-trivial MVDs that violate 4NF.

8) Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

A Join Dependency (JD) occurs when a relation can be reconstructed by joining two or more smaller relations without losing information.

For 5NF, a relation should not contain a non-trivial join dependency unless the involved decompositions are implied by candidate keys.

Example

Consider:

SUPPLY (Supplier, Part, Project)

Where:

- **Supplier** = supplier providing the part
- **Part** = part supplied
- **Project** = project for which the part is supplied

Suppose we have:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2
S2	P1	J1
S2	P1	J2

There can be redundancy because independent supplier-part and part-project relationships are represented repeatedly in the three-column relation.

Step 1: Identify the Join Dependency

The relation can be decomposed into:

SP (Supplier, Part)

SJ (Supplier, Project)

PJ (Part, Project)

The join dependency is:

*** (SP, SJ, PJ)**

Meaning:

SUPPLY=SP⋈SJ⋈PJ

Step 2: Why Redundancy Occurs

Suppose:

S1 supplies P1

S1 works on J1

P1 is required for J1

These facts can combine to produce a tuple in the original SUPPLY relation.

If many suppliers, parts and projects are related independently, the same information can appear in multiple combinations.

This creates join dependency redundancy.

Step 3: Decompose into 5NF

Decompose:

SUPPLY (Supplier, Part)

SUPPLIER_PROJECT (Supplier, Project)

PART_PROJECT (Part, Project)

CREATE AND INSERT DATA

```
mysql> CREATE TABLE Supplier_Part (  
->     Supplier VARCHAR(10),  
->     Part VARCHAR(10),  
->     PRIMARY KEY (Supplier, Part)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Supplier_Part  
-> VALUES  
-> ('S1', 'P1'),  
-> ('S1', 'P2'),  
-> ('S2', 'P1');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Supplier_Project (  
->     Supplier VARCHAR(10),  
->     Project VARCHAR(10),  
->     PRIMARY KEY (Supplier, Project)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Supplier_Project  
-> VALUES  
-> ('S1', 'J1'),  
-> ('S1', 'J2'),  
-> ('S2', 'J1'),  
-> ('S2', 'J2');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```

mysql> CREATE TABLE Supply (
  ->     Supplier VARCHAR(10),
  ->     Part VARCHAR(10),
  ->     Project VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Supply
  -> VALUES
  -> ('S1', 'P1', 'J1'),
  -> ('S1', 'P1', 'J2'),
  -> ('S1', 'P2', 'J1'),
  -> ('S1', 'P2', 'J2'),
  -> ('S2', 'P1', 'J1'),
  -> ('S2', 'P1', 'J2');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Supply;
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P1   | J2      |
| S1       | P2   | J1      |
| S1       | P2   | J2      |
| S2       | P1   | J1      |
| S2       | P1   | J2      |
+-----+-----+-----+
6 rows in set (0.00 sec)

```

```

mysql> CREATE TABLE Part_Project (
  ->     Part VARCHAR(10),
  ->     Project VARCHAR(10),
  ->     PRIMARY KEY (Part, Project)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Part_Project
  -> VALUES
  -> ('P1', 'J1'),
  -> ('P1', 'J2'),
  -> ('P2', 'J1'),
  -> ('P2', 'J2');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

```

```
mysql> SELECT * FROM Supplier_Part;
```

Supplier	Part
S1	P1
S1	P2
S2	P1

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Supplier_Project;
```

Supplier	Project
S1	J1
S1	J2
S2	J1
S2	J2

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Part_Project;
```

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

```
mysql> SELECT SP.Supplier,  
->          SP.Part,  
->          PP.Project  
-> FROM Supplier_Part SP  
-> JOIN Part_Project PP  
-> ON SP.Part = PP.Part  
-> JOIN Supplier_Project SJ  
-> ON SP.Supplier = SJ.Supplier  
-> AND PP.Project = SJ.Project;
```

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2
S2	P1	J1
S2	P1	J2

```
6 rows in set (0.00 sec)
```

Justification

The decomposition is justified because the three relationships are independent and can be stored separately. Joining the decomposed relations produces the original valid tuples, so the decomposition achieves 5NF (Project-Join Normal Form) without unnecessary redundancy.

9) Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Consider this poorly designed college relation:

```
COLLEGE (  
  Student ID,  
  Student Name,  
  Course ID,  
  Course Name,  
  Instructor ID,  
  Instructor Name,  
  Department  
)
```

Example:

Student ID	Student Name	Course ID	Course Name	Instructor ID	Instructor Name	Department
101	Rahul	C01	DBMS	I01	Kumar	CSE
102	Priya	C01	DBMS	I01	Kumar	CSE
103	Arun	C02	Python	I02	Priya	CSE
104	Sneha	C03	Java	I03	Ravi	IT

PART B — FUNCTIONAL DEPENDENCIES

The functional dependencies are:

Student ID → Student Name

Course ID → Course Name, Instructor ID

Instructor ID → Instructor Name, Department

Therefore:

Course ID → Instructor ID → Instructor Name, Department

This creates a transitive dependency.

The candidate key for the original relation is:

(Student ID, Course ID)

because a student can take multiple courses.

NORMALIZE TO 3NF

We remove the dependencies causing redundancy.

Relation 1 — STUDENT

STUDENT (Student ID, Student Name)

Primary Key:

Student ID

Relation 2 — COURSE

COURSE (Course ID, Course Name, Instructor ID)

Primary Key:

Course ID

Relation 3 — INSTRUCTOR

INSTRUCTOR (Instructor ID, Instructor Name, Department)

Primary Key:

Instructor ID

Relation 4 — ENROLLMENT

ENROLLMENT (Student ID, Course ID)

Composite Primary Key:

(Student ID, Course ID)

This table connects students and courses.

```
Database changed
mysql> CREATE TABLE College (
  ->     StudentID INT,
  ->     StudentName VARCHAR(50),
  ->     CourseID VARCHAR(10),
  ->     CourseName VARCHAR(50),
  ->     InstructorID VARCHAR(10),
  ->     InstructorName VARCHAR(50),
  ->     Department VARCHAR(30),
  ->     PRIMARY KEY (StudentID, CourseID)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO College
  -> VALUES
  -> (101, 'Rahul', 'C01', 'DBMS', 'I01', 'Kumar', 'CSE'),
  -> (102, 'Priya', 'C01', 'DBMS', 'I01', 'Kumar', 'CSE'),
  -> (103, 'Arun', 'C02', 'Python', 'I02', 'Priya', 'CSE'),
  -> (104, 'Sneha', 'C03', 'Java', 'I03', 'Ravi', 'IT');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM College;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | CourseID | CourseName | InstructorID | InstructorName |
| Department |
+-----+-----+-----+-----+-----+-----+
| 101 | Rahul | C01 | DBMS | I01 | Kumar | CSE |
| 102 | Priya | C01 | DBMS | I01 | Kumar | CSE |
| 103 | Arun | C02 | Python | I02 | Priya | CSE |
| 104 | Sneha | C03 | Java | I03 | Ravi | IT |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```



```
mysql> CREATE TABLE Student (  
->     StudentID INT PRIMARY KEY,  
->     StudentName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Student  
-> VALUES  
-> (101, 'Rahul'),  
-> (102, 'Priya'),  
-> (103, 'Arun'),  
-> (104, 'Sneha');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Instructor (  
->     InstructorID VARCHAR(10) PRIMARY KEY,  
->     InstructorName VARCHAR(50),  
->     Department VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Instructor  
-> VALUES  
-> ('I01', 'Kumar', 'CSE'),  
-> ('I02', 'Priya', 'CSE'),  
-> ('I03', 'Ravi', 'IT');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Course (  
->     CourseID VARCHAR(10) PRIMARY KEY,  
->     CourseName VARCHAR(50),  
->     InstructorID VARCHAR(10),  
->     FOREIGN KEY (InstructorID)  
->         REFERENCES Instructor(InstructorID)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Course  
-> VALUES  
-> ('C01', 'DBMS', 'I01'),  
-> ('C02', 'Python', 'I02'),  
-> ('C03', 'Java', 'I03');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Enrollment (
->     StudentID INT,
->     CourseID VARCHAR(10),
->     PRIMARY KEY (StudentID, CourseID),
->     FOREIGN KEY (StudentID)
->         REFERENCES Student(StudentID),
->     FOREIGN KEY (CourseID)
->         REFERENCES Course(CourseID)
-> );
```

Query OK, 0 rows affected (0.04 sec)

```
mysql> INSERT INTO Enrollment
-> VALUES
-> (101, 'C01'),
-> (102, 'C01'),
-> (103, 'C02'),
-> (104, 'C03');
```

Query OK, 4 rows affected (0.01 sec)

Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Student;
```

StudentID	StudentName
101	Rahul
102	Priya
103	Arun
104	Sneha

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM Instructor;
```

InstructorID	InstructorName	Department
I01	Kumar	CSE
I02	Priya	CSE
I03	Ravi	IT

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM Course;
```

CourseID	CourseName	InstructorID
C01	DBMS	I01
C02	Python	I02
C03	Java	I03

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM Enrollment;
```

StudentID	CourseID
101	C01
102	C01
103	C02
104	C03

4 rows in set (0.00 sec)

```
mysql> SELECT
->     S.StudentID,
->     S.StudentName,
->     C.CourseID,
->     C.CourseName,
->     I.InstructorID,
->     I.InstructorName,
->     I.Department
-> FROM Student S
-> JOIN Enrollment E
->     ON S.StudentID = E.StudentID
-> JOIN Course C
->     ON E.CourseID = C.CourseID
-> JOIN Instructor I
->     ON C.InstructorID = I.InstructorID;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | CourseID | CourseName | InstructorID | InstructorName |
| Department |
+-----+-----+-----+-----+-----+-----+
| 101 | Rahul | C01 | DBMS | I01 | Kumar |
| CSE |
| 102 | Priya | C01 | DBMS | I01 | Kumar |
| CSE |
| 103 | Arun | C02 | Python | I02 | Priya |
| CSE |
| 104 | Sneha | C03 | Java | I03 | Ravi |
| IT |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

10) Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Example

Relation:

R (A, B, C, D)

Functional dependencies:

A → B

B → C

C → D

1. Find A +

Start:

A += {A}

Using A → B:

{A, B}

Using $B \rightarrow C$:

$\{A, B, C\}$

Using $C \rightarrow D$:

$A^+ = \{A, B, C, D\}$

Since A^+ contains all attributes, A is a candidate key.

2. Find B^+

$B^+ = \{B\}$

Using $B \rightarrow C$:

$\{B, C\}$

Using $C \rightarrow D$:

$B^+ = \{B, C, D\}$

It does not contain A, so B is not a candidate key.

3. Find C^+

$C^+ = \{C, D\}$

Not all attributes $\rightarrow$ not a key.

4. Find D^+

$D^+ = \{D\}$

Not all attributes $\rightarrow$ not a key.

Final Answer

Attribute set	Closure	Candidate Key?
A	$\{A, B, C, D\}$	Yes
B	$\{B, C, D\}$	No
C	$\{C, D\}$	No
D	$\{D\}$	No

Candidate Key = A

Justification

The closure of an attribute set contains all attributes that can be functionally determined from it using the given FDs. Since A^+ contains every attribute of the relation and no proper subset of A exists, A is the only candidate key.